Semana 6: Fundamentos de NAC y Cortafuegos

Asegurando los bordes y el principio de "mínimo privilegio" mediante simulaciones de software.

- **Asignatura:** Comunicación y Seguridad
- **Instructor:** Mg. Gabriel Guerrero Arellano
- **Fecha:** 6 de Octubre de 2026



Ruta de Aprendizaje de Hoy

Estructura de la Sesión:

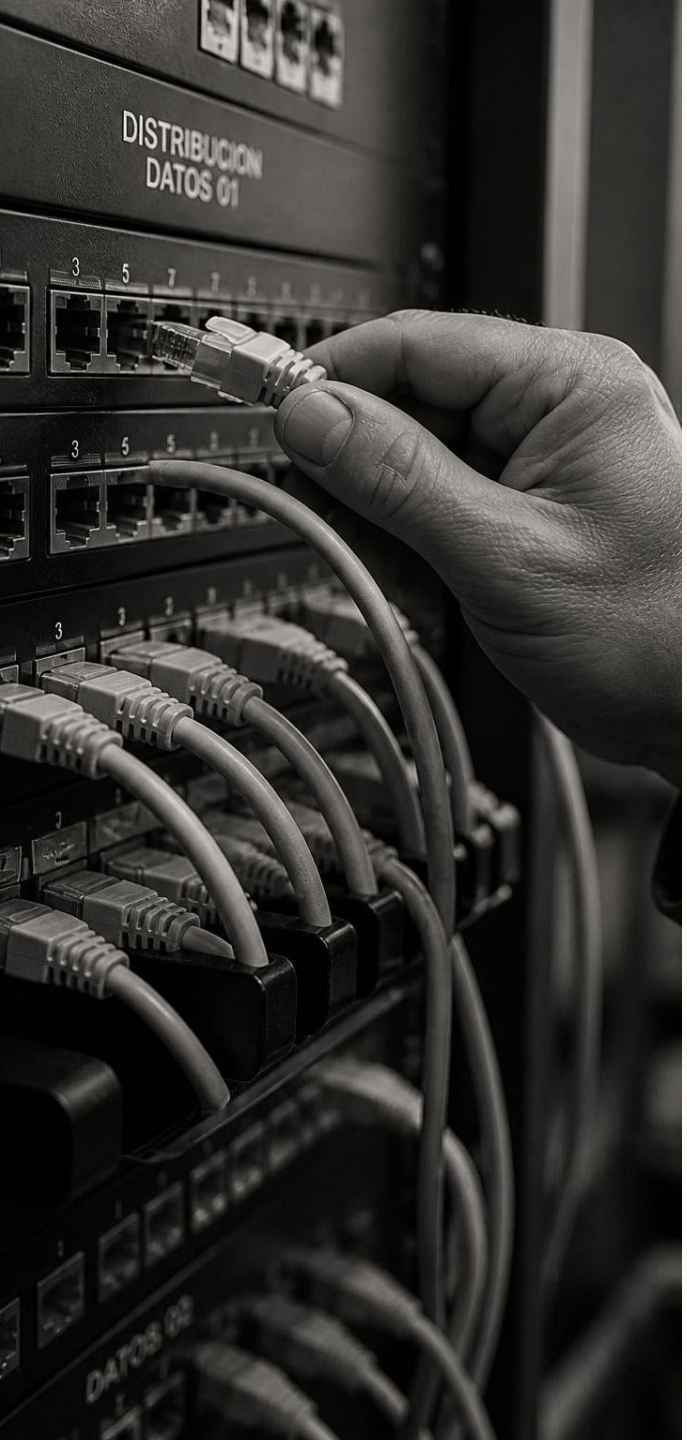
01 **Conexión con clases anteriores:** Del TCP/IP y amenazas a la Tabla de Símbolos.

02 **Conceptos Base:** Network Access Control (NAC) y Fundamentos de Firewalls.

03 **Herramientas:** Modelando la seguridad física con Python.

04 **Laboratorio:** Programación y validación del motor de control de acceso.

05 **Evidencia y Conclusiones.**



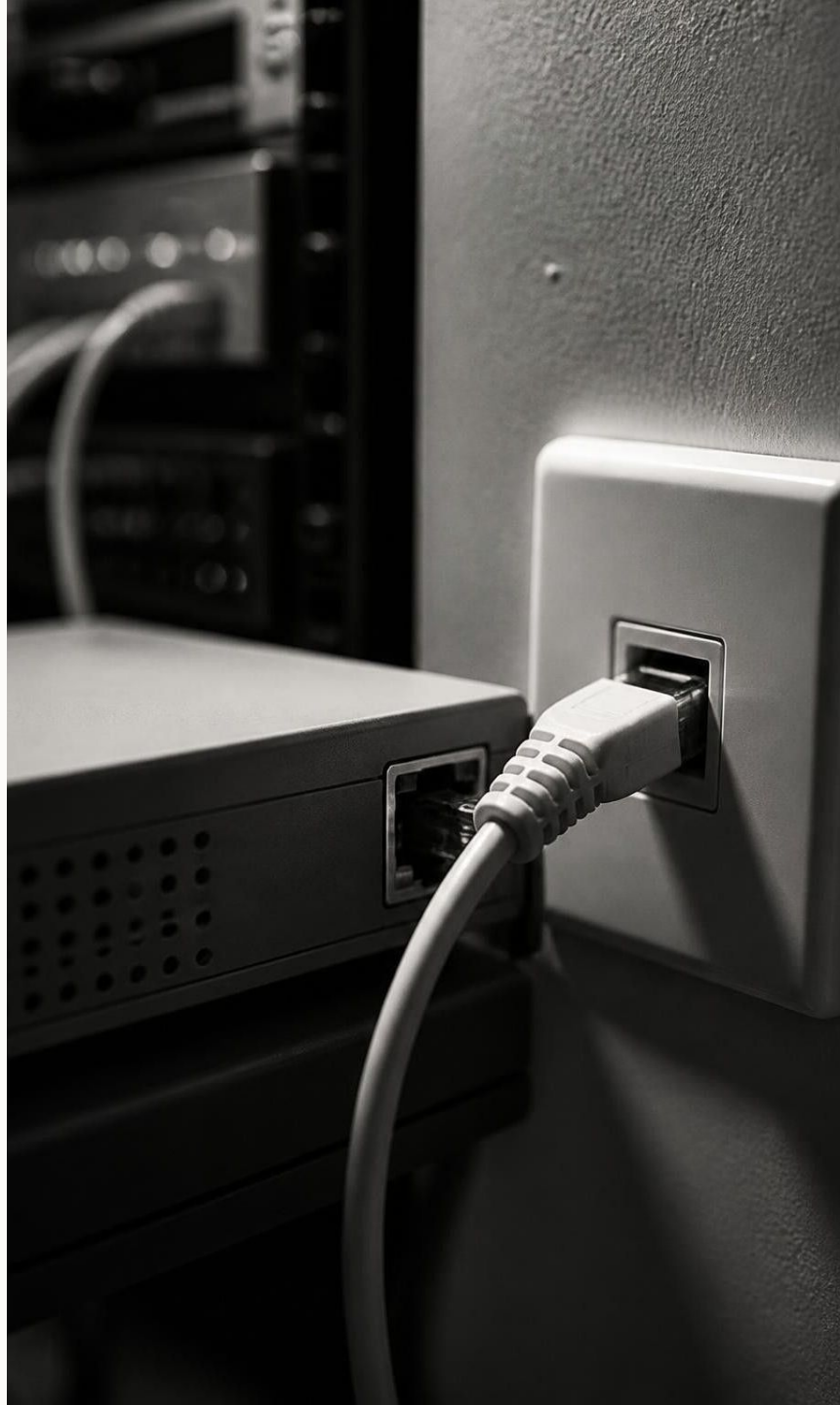
¿Dónde estamos y por qué necesitamos control?

Recap (Semanas 1-5):

- Entendimos la arquitectura **TCP/IP** (Capa 3 y 4: Direcciones IP y Puertos).
- Analizamos amenazas: Criptografía (vulnerable si no se gestiona bien), ataques **MITM** (intercepción) y **DoS** (agotamiento).
- Hicimos Hardening en servidores linux que son medidas sencillas de implementar pero que ayudan a mitigar el espectro de ataque que pueden tener los ciberdelincuentes o hackers.

¿Qué es NAC y por qué es crucial?

- **Definición:** Metodología que gestiona la autenticación, autorización y estado de salud de los dispositivos que solicitan acceso a la red.
- **Objetivo Macro:** Asegurar que solo usuarios/dispositivos verificados y autorizados puedan ingresar a segmentos específicos de la red.
- **Visión Macro:** NAC decide *quién*, *qué* y *dónde* puede conectarse.





El Rol del Cortafuegos en la Red

- **Definición:** Sistema de seguridad de red que monitorea y controla el tráfico de red entrante y saliente basado en reglas de seguridad predeterminadas.
- **Analogía:** Un guardaespaldas o inspector en la frontera, revisando cada paquete que intenta cruzar para ver si cumple con la lista de "permitidos" o "denegados".
- **Estado:** Entiende el contexto (Stateful Inspection), no solo paquetes individuales.

El Mapa de Ámbitos Físicos

Visualización de Zonas:

ZONA EXTERNA (Untrusted):

Internet. El origen de las amenazas.

DMZ (Zona Desmilitarizada):

Zona buffer para servicios públicos (ej: servidor web) que deben ser accesibles desde fuera pero estar aislados de la red interna.

ZONA INTERNA (Trusted):

Dispositivos y servidores críticos de la organización.

¿Qué estamos filtrando exactamente?



Capa 3/4 (Network/Transport - Hoy):

Filtra por direcciones IP (origen/destino) y números de puerto/protocolo (origen/destino). Ejemplo: "Permitir TCP al puerto 80".



Capa 7 (Application):

Entiende protocolos de aplicación (ej: bloquear URLs específicas, patrones de ataques SQLi en HTTP).

Entorno de Trabajo Cero Complicaciones (Sin ISO o Virtual Machine)

- **Entorno:** Vs Code y terminal con Python 3 instalado.
- **Concepto:** Para evitar las fallas anteriores, hoy no configuraremos software de red real. Programaremos un **Motor de Decisión de Control de Acceso**.

Similitud con Compiladores:



Símbolo (Symbol): Representa una regla (ej: puerto, protocolo).



Ámbito (Scope): Representa una Zona de Red (ej: GLOBAL, DMZ, INTERNAL).

La Simplicidad de Nuestra Regla

Sintaxis intuitiva modelada en Python.

Comandos de la Simulación:

`acm.define("80", "TCP", action="ALLOW")`: Permite el puerto 80.

`acm.check_access("80", "TCP")`: Consulta si el puerto 80 está permitido.

`acm.enter_zone("DMZ")`: Crea un nuevo ámbito anidado.

Modelando la Seguridad de un Servidor

Objetivo Práctico: Programar el motor de control de acceso para asegurar un servidor web bajo el **Principio de Mínimo Privilegio**.

Requerimientos de Negocio:

01

Permitir acceso público a la web (HTTP, puerto 80, TCP) en la zona GLOBAL.

02

Permitir acceso administrativo remoto (SSH, puerto 22, TCP) solo desde una zona 'ADMIN' restringida.

03

Denegar *todo* el demás tráfico al servidor (Default Deny).

Paso 1 crear archivo motor_control.py — Estructura de Datos (El Mapa)

```
class Rule:
    """Representa una regla de control de acceso (símbolo)."""
    def __init__(self, port, protocol, action="DENY"):
        self.port = str(port)       # Ej: '80'
        self.protocol = protocol.upper() # Ej: 'TCP'
        self.action = action.upper() # Ej: 'ALLOW' u 'DENY'
class Zone:
    """Representa un ámbito de red o zona de seguridad (scope)."""
    def __init__(self, name, enclosing_zone=None):
        self.name = name                    # Ej: 'GLOBAL', 'DMZ'
        self.rules = {}                     # Reglas locales: {puerto+protocolo: Rule}
        self.enclosing_zone = enclosing_zone # Puntero a la zona superior
```

Paso 2 — Lógica de Búsqueda Escalada de Reglas

```
class AccessControlMotor:
    """Gestor principal de la Tabla de Reglas (firewall simulado)."""
    def __init__(self):
        # Al inicio, el firewall está en la Zona GLOBAL (Default Deny)
        self.current_zone = Zone("GLOBAL")
    def enter_zone(self, name):
        new_zone = Zone(name, enclosing_zone=self.current_zone)
        self.current_zone = new_zone
    def define(self, port, protocol, action="ALLOW"):
        rule = Rule(port, protocol, action)
        self.current_zone.rules[f"{port}/{protocol.upper()}"] = rule
    def lookup(self, port, protocol):
        """ALGORITMO DE BÚSQUEDA ESCALADA DE REGLAS (Herencia de Scopes)."""
        key = f"{str(port)}/{protocol.upper()}"
        zone = self.current_zone
        while zone:
            if key in zone.rules:
                return zone.rules[key] # Regla encontrada
            zone = zone.enclosing_zone  # Subir a zona superior
        return None # No se encontró regla
```

Paso 3 — Verificación de Visibilidad y Aislamiento

```
    def check_access(self, port, protocol):
        """Verifica si se permite el acceso al par puerto/protocolo."""
        rule = self.lookup(port, protocol)
        if rule and rule.action == "ALLOW":
            print(f"  ✓ [PERMITIDO] Tráfico entrante a {port}/{protocol} en '{self.current_zone.name}'.")
        else:
            # Comportamiento por defecto: Deny incoming
            print(f"  ✗ [BLOQUEADO] Tráfico entrante a {port}/{protocol} por POLÍTICA POR DEFECTO.")
# Simulación (en la misma terminal):
if __name__ == "__main__":
    acm = AccessControlMotor()
    # 1. Zona GLOBAL
    acm.define("80", "TCP", action="ALLOW") # Web Público
    acm.check_access("80", "TCP")       # OK (Local)
    # 2. Zona Anidada (INTERNAL)
    acm.enter_zone("INTERNAL")
    acm.check_access("80", "TCP")       # OK (Heredado de GLOBAL)
    acm.define("3306", "TCP", action="ALLOW") # MySQL Interno
    # 3. Zona Anidada (ADMIN_SSH)
    acm.enter_zone("ADMIN_SSH")
    acm.define("22", "TCP", action="ALLOW") # SSH Local
    acm.check_access("80", "TCP")       # OK (Heredado de GLOBAL)
    acm.exit_zone()
    # 4. Retorno a 'GLOBAL' e intento de uso inválido
    acm.check_access("3306", "TCP") # ¡BLOQUEADO! '3306/TCP' murió al salir de la zona INTERNAL
```

Comando de Ejecución del Laboratorio

1. **Comando de Ejecución:** Una vez ubicados en la carpeta donde guardaron el archivo en su terminal o consola (Windows o Linux), deben escribir:

```
python motor_control.py
```

QUE ACABAMOS DE HACER: acabamos de programar un motor que decide qué tráfico entra o no a nuestra red virtual. No estamos usando hardware real, pero la lógica es exacta a la que usa un cortafuegos físico como CISCO o Fortinet[cite: 5]. Lo que ven en la consola no son solo prints, son decisiones de seguridad basadas en el principio de **Mínimo Privilegio** y **Zonas de Red** anidadas

1️⃣ Análisis Paso a Paso de la Ejecución (La Consola)

Vamos a explicar qué ocurrió internamente en cada línea de la salida que obtuvieron al correr `motor_control.py`.

```
[*] Firewall inicializado en zona: GLOBAL
```

- **Explicación:** El cortafuegos arranca en la "raíz" del programa. En este nivel, por política por defecto (NAC básico), **todo tráfico entrante está bloqueado** a menos que se cree una regla explícita.

```
[REGLA AÑADIDA] ALLOW 80/TCP guardada en zona 'GLOBAL'
  ✓ [PERMITIDO] Tráfico entrante a 80/TCP en 'GLOBAL'.
```

- **Lo que ocurrió:** Registramos la regla `80/TCP` como "permitida" en la tabla de símbolos del ámbito `GLOBAL`. Al consultar la visibilidad, el motor la busca localmente en `GLOBAL`, la encuentra con acción `ALLOW` y confirma el acceso.
- **Analogía Real:** Acabamos de abrir el acceso público a nuestro sitio web desde Internet.

1️⃣ Análisis Paso a Paso de la Ejecución (continuación)

```
---> Entrando a zona de seguridad: 'DMZ_SERVICIOS_INTERNOS'
  ✓ [PERMITIDO] Tráfico entrante a 80/TCP en 'DMZ_SERVICIOS_INTERNOS'.
[REGLA AÑADIDA] ALLOW 3306/TCP guardada en zona 'DMZ_SERVICIOS_INTERNOS'
  ✓ [PERMITIDO] Tráfico entrante a 3306/TCP en 'DMZ_SERVICIOS_INTERNOS'.
```

- **Lo que ocurrió (Crítico):**

1. El puntero **current_zone** se mueve a una zona hija (**DMZ**), que anida (contiene) a **GLOBAL**.
2. **Visibilidad Heredada:** Consultamos el puerto **80/TCP**. El motor lo busca en **DMZ** y no lo halla; **sube el escalón hacia GLOBAL** (su zona padre), lo encuentra y valida el acceso. **Demostramos que las zonas hijas heredan la visibilidad de sus padres.**
3. Registramos la regla **3306/TCP** (Base de Datos MySQL) *solo* dentro de la **DMZ**. Desde *adentro* de la **DMZ**, el acceso es correcto (**✓ [PERMITIDO]**).

2️⃣ El Gran Clímax: El Error Semántico de Seguridad (El Bloqueo Final ⭐)

<--- Saliendo de zona de seguridad: 'DMZ_SERVICIOS_INTERNOS'
✗ [BLOQUEADO] Tráfico entrante a 3306/TCP por POLÍTICA POR DEFECTO.

Aquí está el "Aha! Moment" de la clase. Esta es la parte más importante del análisis:

- **Lo que ocurrió:**

1. Ejecutamos **exit_zone()**. El puntero **current_zone vuelve a apuntar a GLOBAL**.
2. El ámbito de visibilidad de la **DMZ** **se destruye o queda inaccesible**. Todas las reglas definidas en la **DMZ** (como **3306/TCP**) mueren para el resto de la red.
3. **La Prueba de Fuego:** Intentamos consultar el acceso a **3306/TCP**. El motor lo busca en la zona actual (**GLOBAL**) → **No existe la regla.**
4. Sube al padre de **GLOBAL** (que es **None**) → **No se encontró regla específica en ningún nivel**.
5. Como no hay regla explícita que diga **ALLOW**, el motor aplica la regla de oro del NAC: **DENY incoming por defecto**.

- **Análisis del Resultado:** Acabamos de demostrar con éxito el **Aislamiento de Zonas**. Aunque la base de datos funciona perfectamente dentro de la **DMZ**, hemos impedido que sea accesible desde Internet (**GLOBAL**). Si un atacante compromete el servidor web en **GLOBAL**, el motor de control ha aislado exitosamente la base de datos en la **DMZ**.

Lo que debe entregar el estudiante

Captura de Pantalla 1: Ejecución completa del script. La consola debe mostrar los mensajes de ✓ **[PERMITIDO]** y, fundamentalmente, la línea final de ✗ **[BLOQUEADO]** del Paso 4.

Pregunta de Reflexión: ¿Cómo demuestra la última línea (✗ **[BLOQUEADO] Tráfico entrante a 3306/TCP**) que el aislamiento de zonas funciona correctamente?

Resumen de Aprendizajes

Resumen de Conclusiones:



Hemos demostrado que los firewalls y el NAC pueden modelarse usando los mismos conceptos de visibilidad de símbolos y ámbitos de visibilidad que los compiladores.



La simulación de software es una herramienta poderosa para validar lógicas de control sin riesgo de bloqueos físicos.



Un firewall efectivo deniega todo por defecto y permite solo lo estrictamente necesario (mínimo privilegio).

Siguiente Clase (Semana 7): Asegurando la Capa de Transporte (TLS) o Políticas Avanzadas de NAC en Entornos Heterogéneos (según syllabus).